

Distributed Systems

Faculty of Informatics, Juraj Dobrila University of Pula

assoc. prof. Nikola Tankovic, PhD

Table of contents

About this book 5

1 Foundations and system models 6
- 1.1 Learning outcomes . . . 6
- 1.2 Example: did my submission succeed? . . . 6
  - 1.2.1 A system boundary . . . 7
- 1.3 What makes a system distributed? . . . 8
  - 1.3.1 Coherence and transparency . . . 9
- 1.4 Must everyone wait for one submission? . . . 9
- 1.5 Concurrency and parallelism . . . 10
- 1.6 Why distribute the work? . . . 11
- 1.7 A remote request has more than a return value . . . 12
- 1.8 Partial failure . . . 14
- 1.9 Two histories behind one timeout . . . 15
  - 1.9.1 Discussion: should the student retry? . . . 16
  - 1.9.2 Reusing concurrency to reason about retries . . . 16
- 1.10 Invariants and progress . . . 17
- 1.11 State the system model . . . 17
  - 1.11.1 Network assumptions . . . 18
  - 1.11.2 Node assumptions . . . 18
  - 1.11.3 Timing assumptions . . . 18
- 1.12 Exercises . . . 19
  - 1.12.1 Suggested reasoning . . . 19
- 1.13 Recap . . . 20
  - 1.13.1 Design the next step: did I submit on time? . . . 20
- 1.14 Sources and further reading . . . 21

2 Architectures and service boundaries 22
- 2.1 Learning outcome . . . 22
- 2.2 Core idea . . . 22
- 2.3 Worked scenario . . . 22
- 2.4 Discussion . . . 22
- 2.5 Recap . . . 22

3 Communication and remote calls 23
- 3.1 Learning outcome . . . 23
- 3.2 Core idea . . . 23
- 3.3 Worked scenario . . . 23

3.4 Discussion . . . 23
3.5 Recap . . . 23

4 Naming and service discovery **24**
4.1 Learning outcome . . . 24
4.2 Core idea . . . 24
4.3 Worked scenario . . . 24
4.4 Discussion . . . 24
4.5 Recap . . . 24

5 Time and event ordering **25**
5.1 Learning outcome . . . 25
5.2 Core idea . . . 25
5.3 Worked scenario . . . 25
5.4 Discussion . . . 25
5.5 Recap . . . 25

6 Replication and consistency **26**
6.1 Learning outcome . . . 26
6.2 Core idea . . . 26
6.3 Worked scenario . . . 26
6.4 Discussion . . . 26
6.5 Recap . . . 26

7 Consensus and leader election **27**
7.1 Learning outcome . . . 27
7.2 Core idea . . . 27
7.3 Worked scenario . . . 27
7.4 Discussion . . . 27
7.5 Recap . . . 27

8 Fault tolerance and recovery **28**
8.1 Learning outcome . . . 28
8.2 Core idea . . . 28
8.3 Worked scenario . . . 28
8.4 Discussion . . . 28
8.5 Recap . . . 28

9 Distributed transactions and workflows **29**
9.1 Learning outcome . . . 29
9.2 Core idea . . . 29
9.3 Worked scenario . . . 29
9.4 Discussion . . . 29
9.5 Recap . . . 29

10 Messaging and stream processing **30**
10.1 Learning outcome . . . 30

10.2 Core idea . . . 30
10.3 Worked scenario . . . 30
10.4 Discussion . . . 30
10.5 Recap . . . 30

11 Partitioning and scalability **31**
11.1 Learning outcome . . . 31
11.2 Core idea . . . 31
11.3 Worked scenario . . . 31
11.4 Discussion . . . 31
11.5 Recap . . . 31

12 Security, observability, and system design **32**
12.1 Learning outcome . . . 32
12.2 Core idea . . . 32
12.3 Worked scenario . . . 32
12.4 Discussion . . . 32
12.5 Recap . . . 32

About this book

This learning book accompanies the proposed Distributed Systems syllabus at the Faculty of Informatics, Juraj Dobrila University of Pula.

Lecturer: assoc. prof. Nikola Tankovic, PhD

Chapter 1 develops the foundations of distributed systems through a university submission example, illustrated explanations, and exercises with suggested reasoning. It accompanies a 60-minute lecture in English.

Chapters 2–12 are short sample placeholders for the planned course sequence. They demonstrate the publishing structure and will be expanded lecture by lecture. They are not yet complete teaching materials.

1 Foundations and system models

1.1 Learning outcomes

After this lecture, you should be able to:

1. Identify the processes, messages, and shared goal in a distributed application.
2. Explain a reason for distribution and the coordination cost it introduces.
3. Distinguish concurrency from parallelism using an execution timeline.
4. Explain partial failure and why a timeout does not establish a remote operation's outcome.
5. State basic network, node, and timing assumptions, and identify a rule the system must always obey.[1]

Prerequisites: basic programming, functions, processes, and an informal understanding of a network. No previous knowledge of distributed algorithms is assumed.

Lecture scope: these foundations support the later lectures on architectures, RPC, clocks, replication, and consensus. We introduce the questions those mechanisms answer; we do not yet implement them. The examples below describe an imagined university project-submission service, not the university's actual infrastructure.

1.2 Example: did my submission succeed?

A student is signed in to a university project-submission portal and already enrolled in the course. For an assignment, they have built a small game in Unity. They package all the project files into a single **ZIP file**: the source code, scenes, and resources such as images and sounds. This **20 MB project archive** is the file they want the teacher to receive. They select it in the portal and press **Submit**. After waiting, the browser displays a timeout. Has the portal accepted the project for the teacher? Should the student submit again?

Uploading bytes and accepting a submission are different steps. Even a progress bar showing 100% does not prove that the portal has validated the rules, saved the complete file, and recorded an accepted submission. In our example, success means that the complete project

[1] Such a rule is called an **invariant**. For example, retrying the same project submission must never create a second accepted entry. Recall **idempotency** from Web Apps: repeating an operation has the same effect as doing it once. Making submission retries idempotent helps preserve this invariant; the reply to each attempt need not be identical.

and its submission record are stored and available for the teacher to retrieve; it does not mean the teacher has opened or graded it.

There are several possible explanations. The complete submission request might never have reached the service; an interrupted upload may leave only temporary bytes. The service might have received it but stopped before storing anything. It might have stored the submission and then lost its reply. Or the request might still be waiting in a queue. The same visible symptom can therefore correspond to different internal states.

This is our central problem for the lecture: **a process must make decisions using incomplete information about other processes**. We will return to the question after building a model of the system. The example develops the uncertain-outcome problem discussed in [3, section 2.1].

1.2.1 A system boundary

The student browser sends the project to a submission service. The **assignment service** supplies assignment details, deadlines, and permitted file types and sizes. The submission service checks those rules and saves the complete project and an accepted-submission record in **submission storage**. We assume the student is already signed in and enrolled, so authentication and enrolment checks are outside this introductory diagram. We include these four cooperating participants in the example's system boundary.

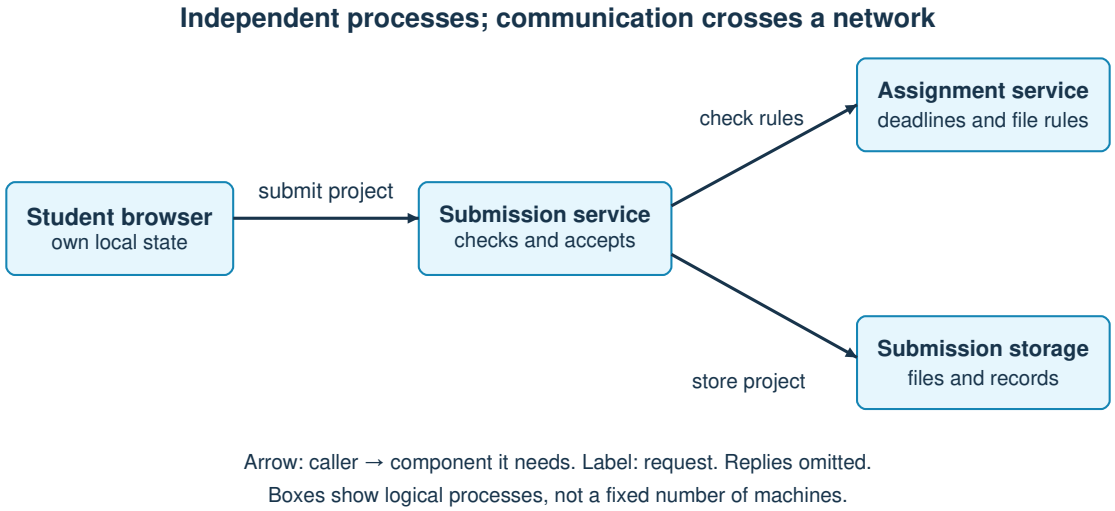


Figure 1.1: The submission example contains separate processes connected by requests. Each box owns local state. An arrow points from a caller to a component it depends on for this operation; its label names the request. Replies are omitted.

A **process** is a running program. A **node** is a unit that we treat as an independent participant in a particular model: it may be a process or a machine. We must say which meaning we intend. Several processes can share one physical host, so a diagram with

four processes does not imply four separate machines or four independent hardware failure domains.

Read each arrow as **"calls and depends on"**: the student browser calls the submission service, and the submission service needs the assignment service to check the rules and submission storage to save the project. The arrow points towards the component being called; it does not mean that information flows only one way. Replies travel back but are omitted from this overview. These are dependencies for the submission operation shown, not a map of every possible dependency in the system.

A submission service cannot simply read a variable in a remote assignment service process as if it were a local variable. In our model, it asks for information by sending a message. Even when an API hides that exchange, information still has to cross a communication boundary.

1.3 What makes a system distributed?

We will use the following definition throughout this lecture.

Definition 1.1

A **distributed system** consists of independently executing components that communicate to carry out a shared task.

The application may aim to present a coherent service to its users, even though the work happens in several places. This separates the structure of the system from a useful design goal: hiding unnecessary details of distribution [3, section 1.1].

Three questions help apply the description:

- **Who acts independently?** Identify the participating processes and their local state.
- **How do they communicate?** Identify requests, replies, events, and the boundaries they cross.
- **What are they trying to achieve together?** State a user-visible operation or shared goal.

A folder containing several unrelated programs is not enough to demonstrate a distributed application. Conversely, a single product name or a single web address does not imply that only one process is involved.

1.3.1 Coherence and transparency

The submission system should offer a meaningful service: students select an assignment and upload a project and obtain an understandable result. They should not need to choose a database machine or understand the assignment service's deployment. **Distribution transparency** means hiding selected implementation details from the user or caller.

Transparency is not a promise that communication has no cost or that failures can always be hidden. For example, replacing an uncertain result with a confident "Submission failed" is misleading if the submission might already exist. A useful interface may instead show "Outcome unknown; checking status" and offer a way to retrieve the result. The policy must match what the system actually knows [2].

We will not require complete transparency for every application. An offline indicator or an explicit synchronization status can be helpful information rather than a design defect.

1.4 Must everyone wait for one submission?

Imagine a large university where many students submit their projects just before the deadline. Receiving the ZIP file is only the first step. The portal checks assignment rules, unpacks and scans the files, saves the project, and records its acceptance. Suppose these steps take **20 minutes for one submission**.

Does this mean nobody else can submit during those 20 minutes? Not necessarily. We need to know how the work is organized:

- With **one worker processing one complete job at a time**, later jobs wait in a queue. With a fixed 20-minute processing time and continuous work, that worker completes three jobs per hour. Other students may still upload their files if receiving uploads is handled separately.
- If much of those 20 minutes is spent **waiting for storage or another service**, a worker may process parts of other submissions during those waits. Several jobs can be in progress without executing instructions simultaneously.
- With **several workers and enough resources**, different projects may be processed at the same time. Two independent workers could ideally complete six such jobs per hour, but only if shared storage, the network, and other checks do not become bottlenecks.

So the time for one submission does not determine how many submissions the whole system can complete per hour. **How can we complete more work in the same amount of time while still accepting each project correctly?** The rate of completed work is called **throughput**. Reducing one student's waiting time is another goal. The next section names these different ways of organizing work.

1.5 Concurrency and parallelism

Let A and B be different students' submissions. A worker can progress B while A waits for storage, or two workers can process A and B simultaneously. These alternatives answer the workload question and motivate the following terms. Their instructions need not execute at the same instant: one worker may switch between them. With two processor cores, both tasks may actually execute at the same time [2].

Definition 1.2

Concurrency means that multiple tasks are in progress during overlapping periods.

Definition 1.3

Parallelism means that multiple tasks execute at the same time.

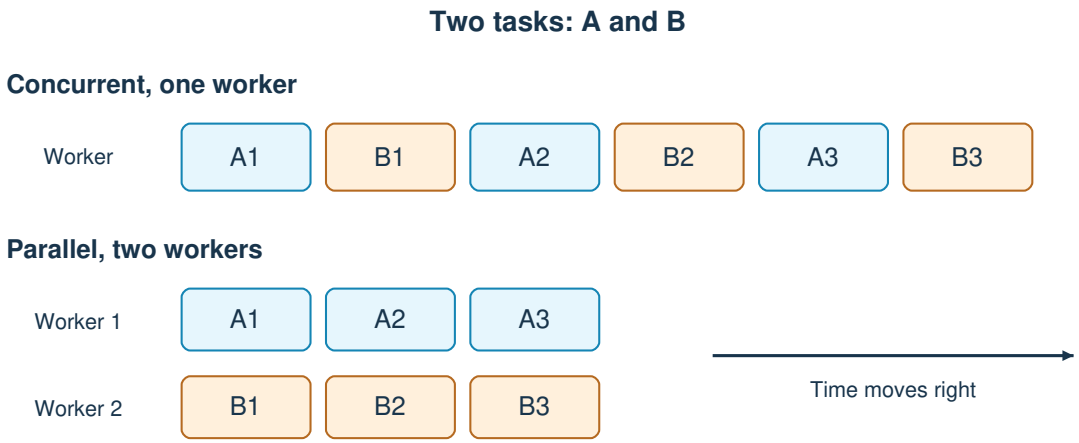


Figure 1.2: The upper timeline interleaves tasks A and B on one worker. The lower timelines execute A and B on separate workers at the same time. Each labelled block is a piece of a task, not necessarily one instruction.

Recall the 20-minute submission checks: that duration forced three jobs per hour only under the assumption of one worker processing jobs strictly one at a time.

Overlapping work can keep resources busy during waits, and parallel execution may increase completed jobs per hour when shared resources keep up. We will later reuse this execution pattern to reason about two attempts of the same submission.

Concurrency can occur inside a single machine; it does not require a distributed system. Distribution adds communication between independently executing participants. Parallelism

can also occur inside a single machine, so multiple cores are not by themselves evidence of a distributed application.

Pause and predict: if job A is waiting for storage, can the worker progress job B? Does this require two processor cores?

1.6 Why distribute the work?

Resource sharing, transparency, openness, and scalability are common goals in distributed-system design; the textbook in [1] offers broader reading on these goals. These goals motivate design choices, but none removes the need to reason about coordination.

Motivation	Submission-service example	New question to answer
Share resources	Several applications use one assignment service.	What happens when that dependency is unavailable?
Complete more work per second	Several workers process different projects.	How do they avoid accepting the same submission twice?
Place work near users	Clients connect to a nearby service instance.	When do updates become visible elsewhere?
Continue through some failures	Another instance can handle requests.	Which state survives, and how does a client find the replacement?
Support organizational boundaries	A faculty assignment service is operated by another team.	Which interface and operational commitments can both sides rely on?

These motivations are broader than speed alone. **Resource sharing** lets multiple courses reuse storage or assignment rules. **Transparency** gives students one coherent portal without requiring them to choose a worker. **Openness** means components follow agreed interfaces so, for example, a faculty tool and the submission portal can cooperate; it does not necessarily mean open-source software. **Scalability** concerns accommodating more work, more locations, or more participating organizations. These are design goals to pursue, not automatic consequences of adding machines [1].

Scaling up means increasing the capacity of an existing component. **Scaling out** means adding components. Scaling out can increase capacity when work can be divided effectively, but a shared bottleneck or additional coordination can limit the benefit. Adding servers does not automatically make an operation correct or faster.

For a small application, one well-understood process and submission storage may be adequate. Distribution becomes justified by a concrete requirement, not by the number of boxes in an architecture diagram. The cost includes communication, deployment, recovery, and reasoning about states that different components can observe at different times.

1.7 A remote request has more than a return value

A function-call interface can hide a remote exchange. Underneath it, a caller prepares a request, the request travels to another process, that process performs work, and a reply travels back. For a worked introduction to this boundary and the conversion between local values and messages, see [3, section 1.3].

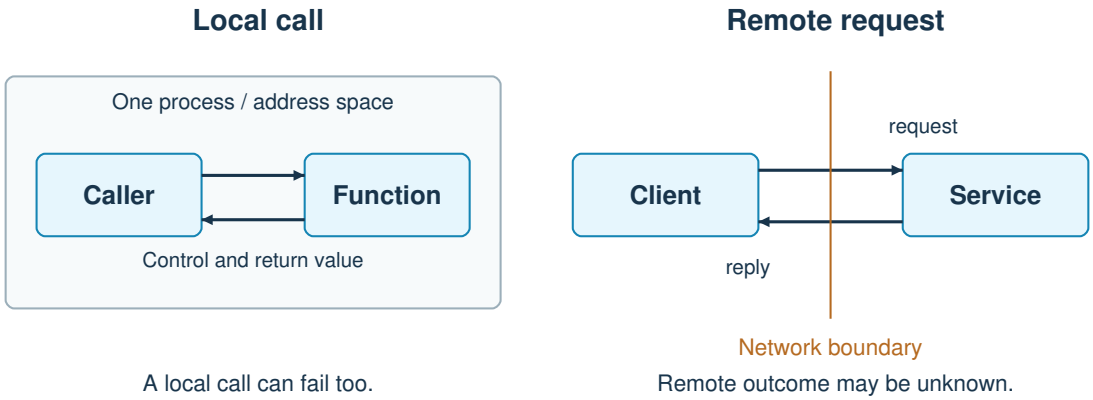


Figure 1.3: A local call transfers control within one process. A remote request crosses a network boundary in both directions, introducing uncertainty about whether work happened when no reply arrives. Local calls can also fail; the distinction is not that they are infallible.

A remote call has several possible costs: request preparation, transmission, queueing, processing, and reply handling. These costs are not necessarily constant. A service that normally responds quickly may pause, become overloaded, or wait for another dependency.

Latency describes a delay, such as the time from issuing a request to receiving a response. **Bandwidth** describes a communication link's data-transfer capacity. **Throughput** is the rate of completed work, such as accepted projects per second. A link with high bandwidth can still have high latency; a service with high throughput can still have slow individual requests. State the measurement and its units rather than treating these terms as interchangeable [3, section 1.2]. A 20 MB archive makes transfer time noticeable, but validation, queueing, and storage can also delay confirmation. The file size is illustrative, not a measured performance claim.

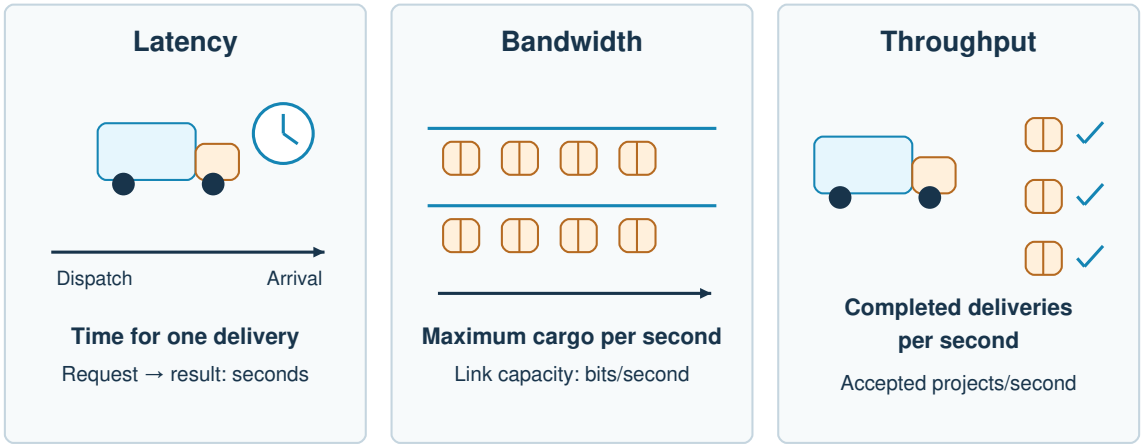


Figure 1.4: A delivery analogy separates the time for one delivery, the route's cargo capacity, and the rate of completed deliveries. These measure different things: a large cargo capacity does not by itself guarantee a short trip or fast acceptance at the destination.

Imagine sending the project as a parcel. **Latency** asks how long this delivery takes; **bandwidth** asks how much cargo the route can carry per second; **throughput** asks how many deliveries are actually completed per second. Cargo volume and completed deliveries are different units. In the portal, we measure latency until the result reaches the student, including the reply. We count throughput here as accepted projects per second. A fast upload can still be followed by a wait for validation or storage: the file arriving is not the same as the project being accepted.

Thinking question: how would you transport **100 TB of data between two datacenters as fast as possible**? What information would you need before choosing a method?

Suggested reasoning: compare sending the data over the network with copying it to drives and transporting them. Define completion as all data copied and verified at the destination. Ask about distance, sustained network transfer rate, available drive capacity, copying speed, and transport time. Using decimal units, 100 TB contains 800 trillion bits; at a sustained 10 Gbit/s, transfer alone takes about **22.2 hours**, before additional overhead. For physical transport, include writing the drives, travel, reading them, and verification. Neither method is always fastest. Moving drives can offer high effective throughput for a large batch while having long delivery latency; it would be a poor way to obtain a quick reply to a small request. These numbers are illustrative, not measured datacenter performance. Returning to the portal: moving lots of data efficiently does not guarantee a quick response to one student.

1.8 Partial failure

An assignment service can stop responding while the submission process, submission storage, and student browser remain alive.

Definition 1.4

A **partial failure** occurs when some components fail while others continue running.

Components need not fail together, and a live process can still be unable to complete a particular operation [2; 3, section 2.3].

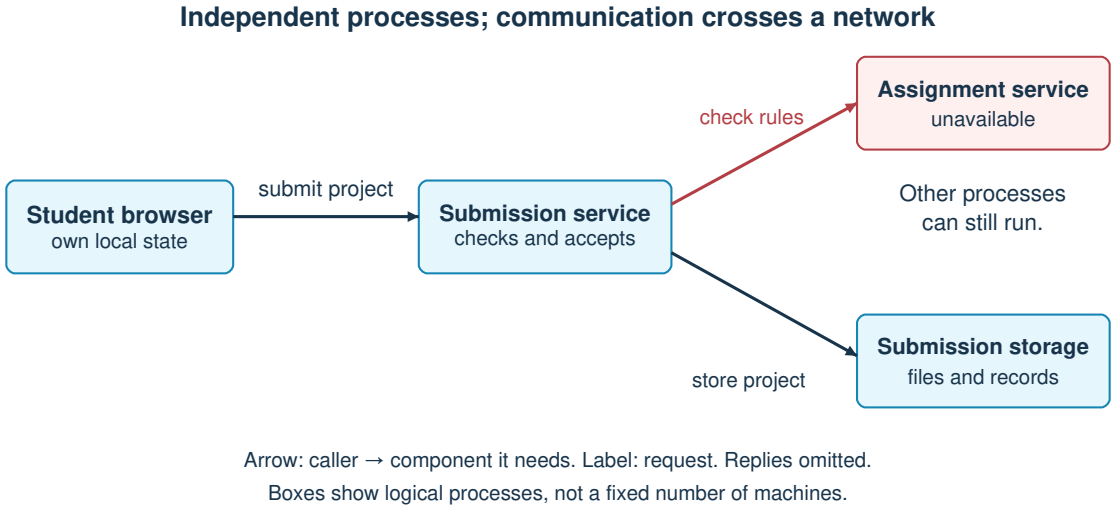


Figure 1.5: The assignment service is unavailable, while the other processes continue running. Whether submission can complete depends on the information and guarantees the operation requires.

"Server is running" and "service operation is available" are different statements. If every submission must check the current deadline and file rules, losing the assignment service can prevent new acceptances even while uploads or receipt retrieval still work. Another operation, such as reading a previously stored receipt, might still work. Availability must be discussed for a particular operation and its required guarantees.

A network interruption can also prevent two live processes from communicating. An observer that receives no reply cannot always tell whether a process stopped, the route failed, or the reply is merely delayed. Failure detection is therefore connected to the assumptions we make about time.

1.9 Two histories behind one timeout

Return to the student's request. We will compare two specific histories, holding the client-visible result constant. The arrows summarize a whole submission operation, not one network packet. History A includes an upload that never completes; temporary file fragments do not count as an accepted submission. History B assumes the complete file and accepted-submission record are stored before confirmation is lost.

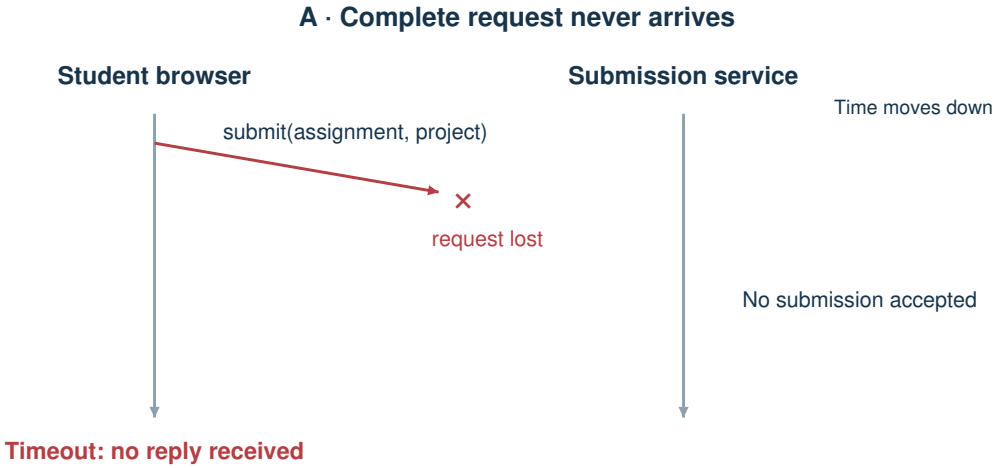


Figure 1.6: History A: the complete submission request never reaches the service. No submission is accepted, and the client eventually reports a timeout.

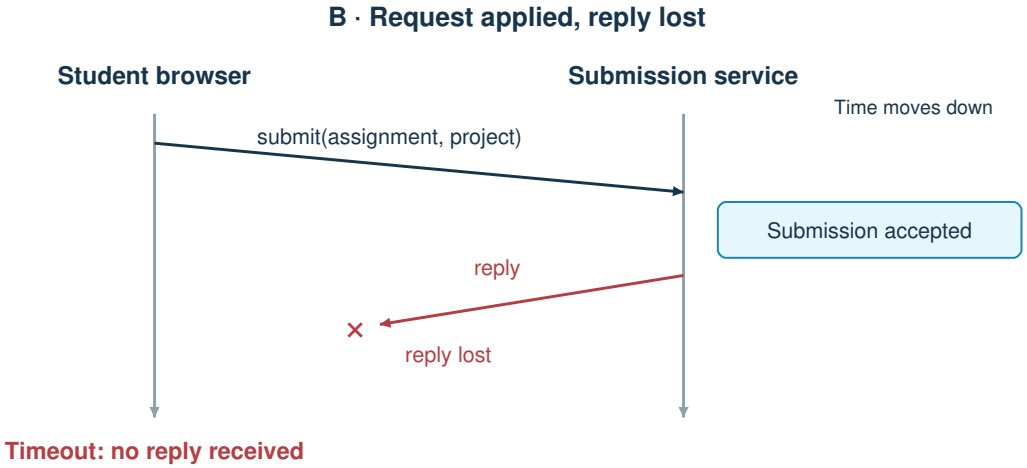


Figure 1.7: History B: the service receives the request and stores the submission, but the reply is lost. The client reports the same timeout as in history A.

In both histories, the client sent a request and did not receive a reply before its deadline. The server-side effect differs. A **timeout is an observation about waiting**, not a definitive statement that a remote effect did or did not happen.

These are not the only histories. A request might still be in flight, queued, or partly processed when the timeout fires. The two examples are sufficient to show why the observation alone cannot distinguish success from failure. They apply the uncertain-outcome concept in [3, section 2.1] to project submission.

1.9.1 Discussion: should the student retry?

A retry could help in history A. In history B, an unprotected retry might repeat an effect, such as creating a second submission entry or notifying the teacher twice. The teacher now has to work out which entry to grade. Whether that happens depends on the operation and the implementation; a timeout does not itself imply duplication.

A better design asks how a client can refer to **the same intended operation** across attempts. For example, a request identifier can be associated with a recorded result. A repeated request can then be recognized. However, generating an identifier alone is insufficient: the receiver must use it consistently, and crashes must not leave the effect recorded without the information needed to recognize it.

We will develop retry semantics, idempotency, and durable duplicate detection in later lectures. For now, retain the design question: **how can the application resolve an uncertain outcome without accidentally repeating an effect?**

1.9.2 Reusing concurrency to reason about retries

A timeout does not cancel work already running at the server. Recall the earlier concurrency diagram, but now let A and B be attempts of the same submission rather than different students' work. Suppose these two attempts represent the same intended project submission. A checks for an existing accepted submission and finds none. Before A records its result, B also checks and finds none. If both then create a new accepted-submission record, the teacher sees two entries for the same intended submission. A retry can overlap the original attempt even after the browser has timed out.

This history does not require simultaneous execution. An interleaving on one worker can expose the same logical problem when an operation spans several steps. We therefore reason about permitted interleavings and shared state, rather than assuming that a fast processor or a single service instance removes the problem.

1.10 Invariants and progress

Definition 1.5

An **invariant** is a rule that must hold in every permitted state of the system.

For the project example, a useful invariant is "retries of the same intended submission create at most one accepted entry." We choose this rule for the example. A deliberate revised project can be a new version under an explicit resubmission policy; it is different from retrying the same operation after a timeout.

Definition 1.6

A **safety property** requires that a specified bad event or state never occurs.

For example, the service must never accept duplicate entries for the same intended submission. Maintaining the one-entry invariant supports this safety property.

Definition 1.7

A **liveness property** requires that specified progress eventually occurs under stated assumptions.

For example, a valid submission eventually gets accepted, provided the necessary services recover and remain available, communication eventually succeeds, pending work is retained or retried, and queued work is eventually processed. Validity follows the applicable assignment rules, including the deadline policy. This is a progress guarantee, not a promise of a particular response time. The exact property and assumptions must always accompany a correctness claim.

A service can avoid duplicates by refusing every submission, but that is not a useful complete solution. It can respond quickly by creating an entry for every request, but retries may then produce duplicates. We need to state both what must not go wrong and the conditions under which useful work must complete.

Before choosing a protocol, write down the invariant, the allowed failures, and the evidence available to each participant. Those three items make an architecture discussion much more precise than a list of technologies.

1.11 State the system model

We have stated the rules to preserve and the progress we want. The next question is: **under what failures and delays can the system provide them?**

A **system model** makes assumptions about participating nodes, communication, and timing explicit. It is an abstraction used to reason about behaviour; it is not a drawing of every implementation detail [3, section 2.3].

1.11.1 Network assumptions

For our worked example, requests and replies can be delayed or lost. The example does not contain an active attacker. Those are choices for the model, not universal facts about every deployment. If we later assume that retransmission eventually succeeds, we must say so; a permanent partition would contradict that assumption.

A delivery guarantee and a timing guarantee are different. "Eventually delivered" does not imply "delivered within two seconds." We should not infer a deadline from a statement that only promises eventual progress.

1.11.2 Node assumptions

Model	What may happen	Consequence for reasoning
Crash-stop	A node stops executing and does not return as that participant.	Other participants cannot rely on further work from it.
Crash-recovery	A node may restart after a crash.	Specify which state is durable and which in-memory state is lost.
Byzantine	A faulty node may deviate arbitrarily from the protocol.	Surviving crashes alone is not sufficient protection.

These categories describe allowed behaviour, not how quickly a failure is detected. In particular, a crash-stop assumption does not automatically give other nodes immediate knowledge of the crash. We do not analyse Byzantine agreement in this introductory lecture.

1.11.3 Timing assumptions

In a **synchronous model**, suitable known bounds constrain communication and processing delays. In an **asynchronous model**, the algorithm cannot rely on such timing bounds. A common **partially synchronous** model allows an initial period without usable timing bounds, followed by a period in which bounds hold; the algorithm may not know when that stable period begins.

The vocabulary here describes guarantees. It does not mean the same thing as a blocking versus non-blocking API. A program using `async` and `await` is not thereby proving that its distributed algorithm uses an asynchronous system model.

A deadline is a policy chosen by an application. It can limit how long a user waits even when it cannot prove that a remote node crashed. If the model supplies dependable bounds, stronger conclusions may be possible. Without those bounds, "usually fast" is not the same as "guaranteed to answer by this time."

Why does "usually" not establish a guarantee? It allows exceptions, while a model guarantee must hold in every execution permitted by that model. Suppose replies usually arrive within 100 ms, but one takes 500 ms. Declaring the server crashed after 100 ms could mistake a slow server for a failed one. The values are illustrative. A correctness argument that depends on a 100 ms upper bound does not cover this slower execution; observations of typical speed do not establish that bound. A model assumption is a basis for reasoning, not evidence that the real system always satisfies it.

1.12 Exercises

1. **Identify the boundary.** In the submission diagram, list each participant and one item of state it owns. Which machine failures could affect more than one participant if two services share a host?
2. **Classify an execution.** Worker W executes A1, B1, A2, B2. Are A and B concurrent? Does this trace demonstrate parallelism? Explain the distinction.
3. **Interpret evidence.** A submission request times out after two seconds. Give two different server-side histories consistent with that observation. Does a longer timeout remove the ambiguity in a model without a known upper delay bound?
4. **Audit an assumption.** A design says, "The assignment service usually responds within 100 ms, so after 101 ms it must have crashed." Identify the missing guarantee and propose a more accurate statement.
5. **Protect a rule.** Two attempts of the same intended submission both check for an existing accepted entry before either creates one. Write an interleaving that produces two entries, state the violated invariant, and explain why adding a second server does not solve the issue by itself.
6. **Choose a tradeoff.** The assignment service is unreachable but cached assignment instructions exist. Discuss when showing those instructions is acceptable and why accepting a new submission may need a stronger guarantee.

1.12.1 Suggested reasoning

1. The client owns its current view and pending request; submission owns processing state; the assignment service owns deadlines and file rules; submission storage owns complete project files and accepted-submission records. A shared host creates a shared failure domain. Exact ownership can differ in a real design and should be stated.
2. The tasks overlap in progress, so the execution is concurrent. The single-worker trace does not show simultaneous execution and therefore does not demonstrate parallelism.

3. The request could be lost, or it could be applied with its reply lost. A larger finite deadline changes the waiting policy but does not supply a bound the model does not guarantee.
4. An observed typical delay is not a guaranteed maximum including network and processing time. "No response by the deadline; suspect unavailability and begin the defined recovery policy" is a defensible observation.
5. A finds no entry, B finds no entry, A creates an accepted entry, then B creates another. The one-entry-per-intended-submission invariant fails. Correctness needs a mechanism that coordinates the check and effect; merely duplicating an uncoordinated implementation leaves the problem.
6. Stale assignment instructions may be acceptable if labelled appropriately. Accepting a submission can depend on the current deadline and permitted file types and sizes. The decision follows the operation's requirements and any explicitly permitted staleness.

1.13 Recap

Distributed components cooperate through communication while retaining independent execution and local state. Distribution can meet useful requirements, but it creates coordination and recovery work. Concurrency does not require parallel hardware. A partial failure can leave some processes alive while an operation becomes unavailable. Most importantly, a timeout does not by itself determine a remote operation's outcome.

For any proposed design, ask: **who participates, what messages cross boundaries, what must remain true, and which assumptions justify the proposed behaviour?**

1.13.1 Design the next step: did I submit on time?

The deadline is **midnight**. A student's upload finishes at **23:59**, but the checks finish at **00:19**. **How would you design the portal so the student knows whether they submitted on time—even after closing the browser?** Propose what the portal should save and what it should show.

First state the deadline policy: which event counts as submission? The browser reporting upload completion is not by itself evidence that the service has durably received the project. Distinguish receiving a project before the deadline from deciding whether it passes the checks.

Directions to explore: a receipt for a stored upload, a server-recorded receipt time, a "received, checks pending" status, and an identifier that lets the student return to the same submission later. Explain what each part of your proposal establishes. More than one design can be defensible; do not assume that a receipt proves the project passed validation.

Extend your design: what if the server crashes during the checks? Which information must survive, how does processing resume, and how can the student retry without creating

another submission? These questions lead into later lectures on service boundaries, durable state, and safe retries.

1.14 Sources and further reading

[1] Maarten van Steen and Andrew S. Tanenbaum. *Distributed Systems, 4th edition*. 2023. Optional broad textbook reading. The authors offer a free personalized PDF through an email-and-CAPTCHA request form; the download link arrives by email. The directly accessible notes in [3] cover the required introductory topics.

[2] Jim Waldo, Geoff Wyant, Ann Wollrath, and Sam Kendall. *A Note on Distributed Computing*. Sun Microsystems Laboratories, 1994. Public author page with the paper attached. Read for the distinction between local and remote computation, including latency, concurrency, and partial failure.

[3] Martin Kleppmann. *Distributed Systems*. University of Cambridge, 2021/22. Public lecture notes, published under CC BY-SA. Read sections 1.1–1.3 for foundations and RPC, 2.1 for uncertainty, and 2.3 for node, network, and timing models. Section 5.1 provides optional follow-up on retries and duplicate effects.

2 Architectures and service boundaries

Sample chapter for the proposed course plan.

2.1 Learning outcome

Identify a service boundary and its communication costs.

Prerequisites: Lecture 01.

2.2 Core idea

Splitting an application into services introduces communication between independently running components. A useful boundary groups related responsibilities, but calls across that boundary add latency and new failure cases.

2.3 Worked scenario

The registration service asks a separate catalogue service whether a course exists. Catalogue unavailability can now affect registration.

2.4 Discussion

Which information could registration cache, and when would that cache become unsafe?

2.5 Recap

Service boundaries should be evaluated against both responsibilities and failure behaviour.

3 Communication and remote calls

Sample chapter for the proposed course plan.

3.1 Learning outcome

Explain why retrying a remote request can duplicate an effect.

Prerequisites: Lectures 01–02.

3.2 Core idea

A remote call can complete even when its response never reaches the caller. Retrying a state-changing operation can repeat its effect unless the receiver detects duplicate requests or the operation is idempotent.

3.3 Worked scenario

A client sends the same registration request twice using one request identifier. The service stores that identifier with the result and returns the stored result for a duplicate.

3.4 Discussion

What must be stored atomically to avoid a duplicate registration after a crash?

3.5 Recap

A retry policy needs an explicit strategy for duplicate effects.

4 Naming and service discovery

Sample chapter for the proposed course plan.

4.1 Learning outcome

Separate a logical service name from a changing network address.

Prerequisites: Lectures 02–03.

4.2 Core idea

A logical service name lets callers refer to a service without embedding the address of a particular process. Discovery maps that name to an address. Cached mappings may become stale when processes move or stop.

4.3 Worked scenario

A client caches the address of the catalogue service. That instance is replaced before the cache entry expires.

4.4 Discussion

How should the client react when its cached address no longer works?

4.5 Recap

Naming hides location changes only when lookup and recovery behaviour are defined.

5 Time and event ordering

Sample chapter for the proposed course plan.

5.1 Learning outcome

Distinguish causal order from wall-clock timestamp order.

Prerequisites: Lectures 01 and 03.

5.2 Core idea

Different machines can report different wall-clock times for related events. Message exchange gives another ordering rule: a message must be sent before that same message is received. This helps reason about causality without assuming perfectly synchronized clocks.

5.3 Worked scenario

The registration service sends a confirmation event. A notification worker receives it and sends an email. The worker clock happens to be behind the service clock.

5.4 Discussion

Can the email have an earlier timestamp without having caused the registration?

5.5 Recap

Reason about causal dependencies before interpreting timestamps.

6 Replication and consistency

Sample chapter for the proposed course plan.

6.1 Learning outcome

Explain how replication lag can produce a stale read.

Prerequisites: Lectures 03 and 05.

6.2 Core idea

Replication stores related state on multiple nodes. With asynchronous replication, an acknowledged update may not yet be visible on every replica. The promised read behaviour must therefore be stated explicitly.

6.3 Worked scenario

A registration is written to a leader. An immediate read from a lagging replica still shows the old course list.

6.4 Discussion

Which routing or synchronization choice could provide read-your-writes behaviour?

6.5 Recap

More copies do not by themselves define what a read is allowed to return.

7 Consensus and leader election

Sample chapter for the proposed course plan.

7.1 Learning outcome

Explain the role of a majority in a three-node consensus group.

Prerequisites: Lectures 05–06.

7.2 Core idea

Consensus lets participating nodes agree on decisions despite some failures. In Raft, a three-node group uses a majority of two. A node isolated from the other two cannot commit a new log entry by itself. See Ongaro and Ousterhout, Raft, sections 2 and 5.

7.3 Worked scenario

Three registration metadata nodes split into a connected pair and one isolated node. The pair can make progress once a suitable leader is established.

7.4 Discussion

Why should the isolated node avoid accepting independent committed updates?

7.5 Recap

Agreement and progress are separate properties; a partition can prevent progress for some nodes.

8 Fault tolerance and recovery

Sample chapter for the proposed course plan.

8.1 Learning outcome

Describe a recovery plan for a crash during request processing.

Prerequisites: Lectures 03 and 07.

8.2 Core idea

Recovery depends on knowing which state survives a failure. A process restart removes volatile state but may preserve durable records. A correct recovery path must reconcile those records with requests that clients may retry.

8.3 Worked scenario

A worker records that a registration task finished, then crashes before acknowledging the queue message. The task is delivered again.

8.4 Discussion

How can the worker recognize that the repeated task was already completed?

8.5 Recap

Design recovery and normal processing together.

9 Distributed transactions and workflows

Sample chapter for the proposed course plan.

9.1 Learning outcome

Identify an invariant that spans two services.

Prerequisites: Lectures 06–08.

9.2 Core idea

A workflow that changes multiple services can stop between steps. An application must decide whether it needs atomic commitment or can expose intermediate states with a recovery or compensation process. Compensation is an application action and is not always an exact undo.

9.3 Worked scenario

A paid course registration reserves a seat and then requests payment. Payment fails after the seat was reserved.

9.4 Discussion

What state should the student see, and how should the reservation be released?

9.5 Recap

Choose coordination mechanisms from the invariant the application must preserve.

10 Messaging and stream processing

Sample chapter for the proposed course plan.

10.1 Learning outcome

Explain why a message consumer may see the same event twice.

Prerequisites: Lectures 03, 05, and 08.

10.2 Core idea

A message broker decouples the time at which a producer sends work from the time at which a consumer processes it. Under at-least-once delivery, a consumer must be prepared to receive a message more than once.

10.3 Worked scenario

A notification consumer sends an email but loses its acknowledgement. The broker redelivers the confirmation event.

10.4 Discussion

Where could duplicate detection live, and what would it need to remember?

10.5 Recap

A delivery guarantee must be connected to the effects performed by the consumer.

11 Partitioning and scalability

Sample chapter for the proposed course plan.

11.1 Learning outcome

Identify how a partition key can create a hotspot.

Prerequisites: Lectures 04, 06, and 10.

11.2 Core idea

Partitioning assigns different data to different nodes. The chosen key affects both query routing and load distribution. Evenly dividing identifiers does not guarantee evenly dividing traffic.

11.3 Worked scenario

Registrations are partitioned by course identifier. One very popular course receives most requests during enrolment.

11.4 Discussion

Why might adding nodes leave the popular course as a bottleneck?

11.5 Recap

Evaluate the request distribution as well as the data distribution.

12 Security, observability, and system design

Sample chapter for the proposed course plan.

12.1 Learning outcome

Justify a design using an invariant, failure scenario, and observable evidence.

Prerequisites: Lectures 01–11.

12.2 Core idea

A system design should state who can perform an action, what must remain true, and how failures will be investigated. Logs and traces help connect operations across service boundaries, but sensitive information should not be copied into diagnostic output unnecessarily.

12.3 Worked scenario

A student reports a missing registration. A request identifier links the API request, durable registration record, and notification event without logging the student password.

12.4 Discussion

Which evidence distinguishes a failed registration from a delayed confirmation?

12.5 Recap

A defensible design connects correctness, access control, and operational evidence.